■ The ML Project Playbook — Living Document

My personal, growing reference for machine-learning projects.

Every time I learn a new technique or make a decision, I add it here.

Next project, I check this FIRST instead of re-deriving everything.

How to use: before each phase of a new project, open the matching section.

After each project, add anything new to the **Techniques Log** and **Scoreboard** at the bottom.

Last updated: 2026-06-26 · Projects logged: India Cancer, P1 Heart Disease

---

■ Contents

---

1. Data Loading & Quality

Step	What to do
Shape & dtypes	`df.shape`, `df.dtypes` — know rows × cols and which are numeric vs object
Peek	`df.head()`, `df.sample(5)`
Missing values	`df.isnull().sum()` — note which columns, how many
Duplicates	`df.duplicated().sum()` — **exclude ID columns** before counting (an ID makes every row unique)
Target balance	`y.value_counts(normalize=True)` — decides metric (balanced → accuracy ok; imbalanced → F1/PR)
Data realism	Are values plausible? Suspiciously uniform counts → likely synthetic data

---

2. Data Cleaning

Missing values — decision order

• **Understand WHY it's missing** (value counts, is it related to another column?)

• **Don't drop rows** unless missingness is tiny AND random.

• **Choose imputation:**

Strategy	Use when
`median`	Numerical, robust to outliers (default for numeric)
`most_frequent`	Categorical
Grouped imputation	When another column predicts the missing one → fill with that group's median/mode
`PowerTransformer`/model-based	Advanced, rarely needed early

■ **Grouped imputation (learned in P1 & Titanic):** fill a missing column using the value typical for a related group.

- Titanic: missing `Age` → median age of each `Title` group (Master = young boys).

- P1 Heart: missing `ca`/`thal` → median/mode of each **age band** (blocked vessels rise with age).

- **Rule:** only helps if the grouping variable actually relates to the missing column — *check the evidence first* (group-wise summary vs global).

- **Build it as a custom transformer inside the Pipeline** so it re-fits per CV fold → no leakage.

Outliers

• Detect with IQR boxplots (`Q1 - 1.5·IQR`, `Q3 + 1.5·IQR`).

• **KEEP** if clinically/physically real (e.g. cholesterol 564 is real). Only remove true data-entry errors.

• If outliers worry you, use `RobustScaler` (median-based) instead of `StandardScaler`.

Data leakage columns (drop these!)

• Any column that wouldn't exist at prediction time.

• India Cancer: dropped `Survival_Months` (encodes time-to-death → fake 97% accuracy).

• **Rule:** if a feature is a *consequence* of the target, it's leakage.

---

3. EDA

Univariate (one variable at a time)

• Histograms — see distribution shape.

• **Skewness** (`.skew()`): |skew| > 1 = highly skewed. **Kurtosis** = tailedness.

• Boxplots — spot outliers.

Bivariate (feature vs target)

• Numerical vs target → boxplots split by class + **t-test**.

• Categorical vs target → bar chart of target rate per category + **chi-square**.

• Numerical vs numerical → scatter + **correlation**.

• **Best practice:** put the plot AND the statistical test result on the same figure (pattern + "is it real?").

Correlation

• **Pearson** = linear relationship. **Spearman** = monotonic (robust to non-linearity).

• Heatmap to spot feature-feature redundancy (multicollinearity).

• ■■ **Sign vs strength:** a correlation of –0.43 is STRONG (negative = direction, not weakness). Only |ρ|≈0 means weak.

---

4. Statistical Analysis

Which test for which data type

Feature	Target	Test	Measures
Categorical	Categorical	**Chi-square**	Are category distributions different?
Numerical	Binary (2 groups)	**t-test**	Are the two means different?

Feature	Target	Test	Measures
Numerical	3+ groups	**ANOVA**	Are means different across groups?
Numerical	Numerical	**Pearson / Spearman**	Do they move together?
Numerical (non-normal)	Categorical	**Mann-Whitney U**	Non-parametric t-test

Always pair significance with effect size

• p-value answers "is it real?" — **effect size** answers "is it big enough to matter?"

• With large N, tiny meaningless differences become "significant." Effect size guards against this.

• Effect sizes: **Cohen's d** (t-test), **η^2 (eta-squared)** (ANOVA), **Cramér's V** (chi-square), **$|\rho|$** (correlation).

Multicollinearity — VIF

• **VIF (Variance Inflation Factor):** how much a feature is explained by the *other* features.

• VIF < 5 = fine; 5–10 = moderate; >10 = serious redundancy → consider dropping one.

5. Feature Engineering

Technique	Example
Extract from dates	`Diagnosis_Date` → year, month
Binning	Age → age groups (captures non-linear effects)
Ordinal scores	Treatment type → aggressiveness score 1–5 (domain knowledge)
Interaction features	`Stage × Cancer_Type` (Stage IV lung ≠ Stage IV breast)
Ratio features	rooms / households
Binary flags	high-risk = (Stage IV AND Palliative) — **only if the group is class-pure**, check scatter first

Rule: features encode domain knowledge; algorithms find patterns. Better features usually beat more tuning.

Zero-inflated columns (big spike at 0): log won't help. A binary "is-zero" flag only helps if the zero group is mostly one class — verify on the scatter (P1 `oldpeak` had both classes at 0 → no flag).

Skewed features — decision

• **Model first:** trees are skew-immune (don't transform); linear models benefit.

• Smooth right tail, all positive → `log1p`. Has zeros/negatives → `PowerTransformer (Yeo-Johnson)`.

• **Measure, don't assume:** compare raw vs transformed by CV. If within ~0.005, keep it simple.

6. Preprocessing & Pipelines

The leakage rule (most important)

Split FIRST. Fit preprocessing on TRAIN only. Transform both.

Anything that *learns a statistic* (imputation median, scaling mean/std, encoder categories) must be fit inside the Pipeline so CV re-fits it per fold.

Safe before split (row-by-row, learns nothing): dropping columns, extracting date parts, ratios.

The reusable skeleton

```
num_pipe = Pipeline([('imputer', SimpleImputer(strategy='median')),
                     ('scaler',  StandardScaler())])
cat_pipe = Pipeline([('imputer', SimpleImputer(strategy='most_frequent')),
                     ('ohe',     OneHotEncoder(handle_unknown='ignore'))])
preprocessor = ColumnTransformer([('num', num_pipe, num_cols),
                                  ('cat', cat_pipe, cat_cols)])
full = Pipeline([('pre', preprocessor), ('clf', LogisticRegression())])
```

Encoding

Method	Use when
`OneHotEncoder`	Low-cardinality categoricals (correct for CV; use `handle_unknown='ignore'`)
`pd.get_dummies`	Quick EDA only — NOT inside CV (no train/test consistency)
Ordinal encoding	Ordered categories (quality: low<med<high)
Target / frequency encoding	High-cardinality (many categories) — *to learn in Telco/Adult*

Scaling

• `StandardScaler` (default), `RobustScaler` (outliers), `MinMaxScaler` (bounded). Trees don't need scaling.

7. Class Imbalance

To be expanded in Telco Churn & CTG projects.

Tool	Idea
`class_weight='balanced'`	Penalize minority errors more (built into many sklearn models)
`scale_pos_weight`	XGBoost equivalent = n_neg / n_pos
SMOTE	Synthesize minority samples (use `imblearn.Pipeline`; fit on TRAIN folds only)
Threshold tuning	Lower the 0.5 cutoff to catch more positives
Metric switch	Use **PR-AUC / macro-F1**, not accuracy (accuracy is misleading when imbalanced)

India Cancer note: SMOTE ≈ no-balancing on weighted-F1 there; `scale_pos_weight` hurt it. **Always test, don't assume.**

8. Model Selection

Always start with a baseline

• `DummyClassifier(strategy='most_frequent')` / `DummyRegressor(strategy='mean')`.

• If you don't clearly beat it, nothing else matters.

Roster by family (small/medium tabular)

Family	Models
Linear	LogisticRegression, Ridge, Lasso, ElasticNet
Distance	KNN
Probabilistic	GaussianNB
Margin	SVM (fine on small data; slow on large)
Single tree	DecisionTree
Bagging	RandomForest, ExtraTrees
Boosting	GradientBoosting, **XGBoost**, **LightGBM**, AdaBoost

• Compare with **StratifiedKFold** CV (classification) / KFold (regression).

• **Key lesson:** on small clean tabular data, **Logistic/Linear often beats boosting**. Complexity must earn its place.

• Exclude SVM/KNN on very large data (too slow).

9. Hyperparameter Tuning

Search	Use when
`GridSearchCV`	Small grid, fast model (exhaustive)
`RandomizedSearchCV`	Large grid / slow model (n_iter samples)

Common knobs

- **LogisticRegression:** `C` (logspace -3→2), `penalty` (l1/l2/elasticnet), `solver` (liblinear/saga), `class_weight`.
- **RandomForest:** `n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`.
- **XGBoost:** `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree`.
- Param naming in pipeline: `clf__C`, `clf__max_depth`.

Honest note: on small clean data, tuning often moves the score by only ±0.01. Confirming the default is good IS a valid result.

■■ **Selection tax:** `grid.best_score_` is optimistically biased (winner's curse). The honest fix is **nested CV**.

---

10. Evaluation & Metrics

Classification

Metric	Use
Accuracy	Only if balanced
Precision / Recall	Recall = catch positives (medical!); Precision = avoid false alarms
F1 / **macro-F1**	Imbalanced; macro-F1 for multiclass (rare classes count equally)
ROC-AUC	Ranking quality, threshold-free (good for model selection)
PR-AUC	Better than ROC when positives are rare
Confusion matrix	See *which* errors
Calibration curve	Do predicted probabilities mean what they say?

Regression

- **RMSE** (penalizes big errors), **MAE** (robust), **R^2** (variance explained).
- Always beat `DummyRegressor(mean)` → $R^2 > 0$.

Decision threshold

- Default 0.5 is arbitrary. Choose deliberately:
- **Youden's J** = argmax(TPR – FPR) (balanced).
- **Cost-based** = pick threshold minimizing business/clinical cost (FN vs FP weighting).

---

11. Rigor & Honesty Checks

The "Chunk 7" template — run these at the end of every project.

- **Distribution, not a point** — `RepeatedStratifiedKFold(5×10)`; report **mean ± std**. A single hold-out (esp. small test set) is one noisy draw.
- **Error bars on the bake-off** — top models within 1 std = statistically tied → pick on interpretability/speed.
- **Right metric + chosen threshold** — report sensitivity/recall, not just accuracy; set the operating point on purpose.
- **Domain check** — what is the model *really* using? (P1: demographics-only AUC ~0.65 vs full ~0.90 → it's a diagnosis aid, not a screener.)
- **Learning curve** — converged gap = data-saturated (more data won't help); open gap = get more data.
- **Validation curve** — plot train vs val over a key param to see the overfit→underfit arc.
- **Distrust too-good scores** — on small/leaky data, >0.95 usually = a bug, not genius.

---

12. Interpretability

Tool	Model	Reading
Coefficients	Linear	+ pushes to class 1, – to class 0; magnitude = strength (standardize first)

Tool	Model	Reading
Feature importance	Trees	impurity-based (biased to high-cardinality)
Permutation importance	Any	shuffle a feature, measure score drop — multivariate, catches interactions
SHAP	Any (TreeExplainer for trees)	per-prediction contributions; use `shap.Explainer` (TreeExplainer breaks on new XGBoost)

Cross-check: do the top features match what EDA/stat tests flagged? If L1 penalty wins, weak features get coefficient = 0 (auto selection).

13. The Golden Rules

- **Split before you learn anything** (impute/scale/encode after split, inside Pipeline).
- **Beat the dumb baseline** or nothing else matters.
- **Right metric** — not just accuracy (macro-F1 / PR-AUC / RMSE).
- **Significance ≠ strength** — pair every p-value with an effect size.
- **Distrust scores that are too good** — near-perfect on small/leaky data = methodology bug.
- **Measure, don't assume** — test transforms/resampling/tuning with CV; keep only real gains.
- **Complexity must earn its place** — prefer the simpler model when scores tie.

14. ■ Techniques Log

Append every new trick here with the project + date it was learned.

Date	Project	Technique	One-line summary
2026-06	India Cancer	Leakage column detection	Dropped `Survival_Months` (consequence of target) → killed fake 97% accuracy
2026-06	India Cancer	Pipeline + ColumnTransformer	OHE + scale as one leakage-safe object
2026-06	India Cancer	SMOTE vs class_weight test	Resampling didn't help weighted-F1 here — always test
2026-06	P1 Heart	Grouped imputation (custom transformer)	Fill `ca`/`thal` by age-band median/mode, leakage-safe
2026-06	P1 Heart	Skew handling decision	log1p vs PowerTransformer; trees are skew-immune
2026-06	P1 Heart	Combined plot + stat test	Bar/box chart with chi-square/t-test annotation on same figure
2026-06	P1 Heart	Permutation importance	Multivariate feature value; beats single correlation bar
2026-06	P1 Heart	Rigor & Honesty chunk	Repeated CV distribution, error bars, threshold, learning/validation curves
2026-06	P1 Heart	Domain check (demographics-only)	Quantify how much skill comes from "downstream" features
	(next)		

15. ■ Scoreboard

Best honest result per project — for sanity-banding future work.

Project	Task	Metric	Score	Baseline	Notes
India Cancer	Binary (Alive/Deceased)	Weighted F1	~0.71	—	After dropping leakage; GB ≈ LR ≈ XGB
P1 Heart Disease	Binary (disease)	ROC-AUC	~0.84–0.90 (full)	Dummy ~0.54	Demographics-only ~0.65–0.70; models statistically tied
Telco Churn	Binary (imbalanced)	PR-AUC / recall	*TBD*		NEXT
Ames Housing	Regression	RMSE / R^2	*TBD*		
Fetal Health (CTG)	Multiclass	macro-F1	*TBD*		
Medical Insurance	Regression	RMSE / R^2	*TBD*		
Covertype	Multiclass (big)	macro-F1	*TBD*		
Energy Efficiency	Multi-output reg	RMSE / R^2	*TBD*		

End of playbook. Keep it close, keep it growing.